

AtliQ Hardware SQL Report

Task 1 - Identify Croma India Customer Code

Business Objective:

Find the customer code of Croma in the Indian market.

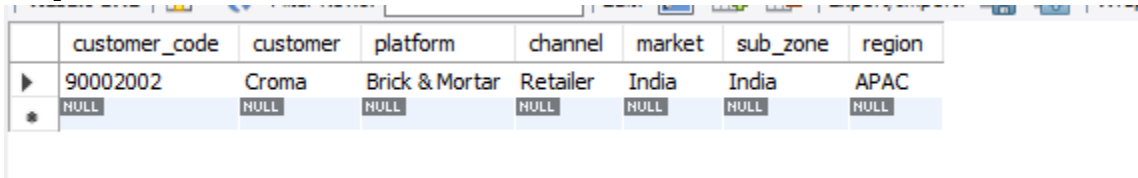
Approach:

Filter the dim_customer master table using customer name and market conditions.

SQL Query:

```
SELECT *  
FROM dim_customer  
WHERE customer LIKE "%croma%" AND market="india"
```

Output Screenshot:



	customer_code	customer	platform	channel	market	sub_zone	region
▶	90002002	Croma	Brick & Mortar	Retailer	India	India	APAC
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL

USER DEFINED FUNCTIONS

Task 2 - Create Fiscal Year Function

Business Objective:

Build a reusable fiscal year calculation function.

Approach:

Create a User Defined Function(UDF) 'get_fisca_year' that converts calendar dates into company fiscal years.

SQL Query:

```
1 CREATE DEFINER=`root`@`localhost` FUNCTION `get_fiscal_year`(  
2   calendar_date DATE )  
3   RETURNS int  
4   DETERMINISTIC  
5   BEGIN  
6     DECLARE fiscal_year INT;  
7     SET fiscal_year = year(DATE_ADD(calendar_date, interval 4 month));  
8     RETURN fiscal_year;  
9   END
```

Using this get_fiscal_year() function, retrieve all sales transactions for Croma India (Customer Code: 90002002) from the fact_sales_monthly table for Fiscal Year 2021.

```
SELECT *  
FROM fact_sales_monthly  
WHERE customer_code = 90002002 AND get_fiscal_year(DATE) =2021  
ORDER BY DATE ASC LIMIT 100000;
```

Output Screenshot:

Since the company's fiscal year starts in September, Fiscal Year 2021 begins on September 1, 2020. Therefore, the output includes all sales transactions from 2020-09-01 onward that belong to Fiscal Year 2021.

	date	fiscal_year	product_code	customer_code	sold_quantity
▶	2020-09-01	2021	A0118150101	90002002	202
	2020-09-01	2021	A0118150102	90002002	162
	2020-09-01	2021	A0118150103	90002002	193
	2020-09-01	2021	A0118150104	90002002	146
	2020-09-01	2021	A0219150201	90002002	149
	2020-09-01	2021	A0219150202	90002002	107
	2020-09-01	2021	A0220150203	90002002	123
	2020-09-01	2021	A0320150301	90002002	146
	2020-09-01	2021	A0321150302	90002002	236
	2020-09-01	2021	A0321150303	90002002	137

les_monthly 1 ×

Task 3 - Create Fiscal Quarter Function

Business Objective:

Generate fiscal quarter wise sales data for Croma .

Approach:

Create a User Defined Function ('get_fiscal_quarter') and use CASE statements to map months into fiscal quarters.

SQL Query:

```

1 • CREATE DEFINER=`root`@`localhost` FUNCTION `get_fiscal_quarter`(
2     calendar_date DATE
3 ) RETURNS char(2) CHARSET utf8mb4
4     DETERMINISTIC
5 BEGIN
6     DECLARE m TINYINT;
7     DECLARE qtr CHAR(2);
8     SET m = MONTH(calendar_date);
9     CASE
10         WHEN m IN (9,10,11) THEN
11             SET qtr = "Q1";
12         WHEN m IN (12,1,2) THEN
13             SET qtr = "Q2";
14
15         WHEN m IN (3,4,5) THEN
16             SET qtr = "Q3";
17         ELSE
18             SET qtr = "Q4";
19     END CASE;
20     RETURN qtr;
21 END

```

Use this `get_fiscal_quarter()` function to retrieve the 'sold_quantity' data for Croma India (Customer Code: 90002002) for Fiscal Year 2021, specifically for Quarter 4 (Q4).

```

1 • SELECT date,sold_quantity
2 FROM fact_sales_monthly
3 WHERE customer_code = 90002002
4 AND get_fiscal_year(date) = 2021
5 AND get_fiscal_quarter(date) = "Q4"
6 ORDER BY date ASC ;

```

Output Screenshot:

	date	sold_quantity
▶	2021-06-01	205
	2021-06-01	78
	2021-06-01	48
	2021-06-01	126
	2021-06-01	40
	2021-06-01	102
	2021-06-01	31
	2021-06-01	91
	2021-06-01	70
	2021-06-01	145

JOINING TABLES

Task 4 - Product-Level Sales Report

Business Objective:

Generate a monthly product-level sales report for Croma India (Customer Code: 90002002).

Approach:

Joining the 'fact_sales_monthly' and 'dim_product' tables to retrieve product details, variants, and sold quantities.

SQL Query:

```
15 • SELECT s.date,p.product_code,p.product,p.variant,s.sold_quantity
16 FROM fact_sales_monthly s
17 JOIN dim_product p
18 ON s.product_code = p.product_code
19 WHERE customer_code = 90002002
20 AND get_fiscal_year(date) = 2021
21 ORDER BY date ASC ;
```

Output Screenshot:

	date	product_code	product	variant	sold_quantity
▶	2020-09-01	A0118150101	AQ Dracula HDD – 3.5 Inch SATA 6 Gb/s 5400 R...	Standard	202
	2020-09-01	A0118150102	AQ Dracula HDD – 3.5 Inch SATA 6 Gb/s 5400 R...	Plus	162
	2020-09-01	A0118150103	AQ Dracula HDD – 3.5 Inch SATA 6 Gb/s 5400 R...	Premium	193
	2020-09-01	A0118150104	AQ Dracula HDD – 3.5 Inch SATA 6 Gb/s 5400 R...	Premium Plus	146
	2020-09-01	A0219150201	AQ WereWolf NAS Internal Hard Drive HDD – 8....	Standard	149
	2020-09-01	A0219150202	AQ WereWolf NAS Internal Hard Drive HDD – 8....	Plus	107
	2020-09-01	A0220150203	AQ WereWolf NAS Internal Hard Drive HDD – 8....	Premium	123
	2020-09-01	A0320150301	AQ Zion Saga	Standard	146
	2020-09-01	A0321150302	AQ Zion Saga	Plus	236
	2020-09-01	A0321150303	AQ Zion Saga	Premium	137

Result 2 x

Task 5 - Gross Sales Report: Product-Level Revenue Analysis for Croma India

Business Objective:

Get product sales details and calculate gross sales for Croma India in FY 2021 Q4.

Approach:

- Join sales(fact_sales_monthly), product(dim_product), and gross price(fact_gross_price) tables to get product details and calculate gross sales amount.
- Used the get_fiscal_year() and get_fiscal_quarter() functions to filter records for Fiscal Year 2021 and Quarter 4.

SQL Query:

```
8 • SELECT
9     s.date,s.product_code,s.sold_quantity,
10    p.product,p.variant,
11    g.gross_price,
12    ROUND(g.gross_price * s.sold_quantity,2) as gross_price_total
13 from fact_sales_monthly s
14 JOIN dim_product P
15 ON s.product_code = p.product_code
16 JOIN fact_gross_price g
17 ON
18     g.product_code = s.product_code and
19     g.fiscal_year = get_fiscal_year(s.date)
20 where
21     customer_code = 90002002 and
22     get_fiscal_year(date) = 2021 and
23     get_fiscal_quarter(date) = "Q4"
24 order by date asc;
25
```

Output Screenshot:

Result Grid Filter Rows: Export: Wrap Cell Content:							
	date	product_code	sold_quantity	product	variant	gross_price	gross_price_tota
▶	2021-06-01	A0118150101	205	AQ Dracula HDD – 3.5 Inch SATA 6 Gb/s 5400 R...	Standard	19.0573	3906.75
	2021-06-01	A0118150102	78	AQ Dracula HDD – 3.5 Inch SATA 6 Gb/s 5400 R...	Plus	21.4565	1673.61
	2021-06-01	A0118150103	48	AQ Dracula HDD – 3.5 Inch SATA 6 Gb/s 5400 R...	Premium	21.7795	1045.42
	2021-06-01	A0118150104	126	AQ Dracula HDD – 3.5 Inch SATA 6 Gb/s 5400 R...	Premium Plus	22.9729	2894.59
	2021-06-01	A0219150201	40	AQ WereWolf NAS Internal Hard Drive HDD – 8....	Standard	23.6987	947.95
	2021-06-01	A0219150202	102	AQ WereWolf NAS Internal Hard Drive HDD – 8....	Plus	24.7312	2522.58
	2021-06-01	A0220150203	31	AQ WereWolf NAS Internal Hard Drive HDD – 8....	Premium	23.6154	732.08
	2021-06-01	A0320150301	91	AQ Zion Saga	Standard	23.7223	2158.73
	2021-06-01	A0321150302	70	AQ Zion Saga	Plus	27.1027	1897.19

Task 6 - Monthly Gross Sales Report for Croma India

Business Objective:

Calculate the monthly gross sales generated by Croma India.

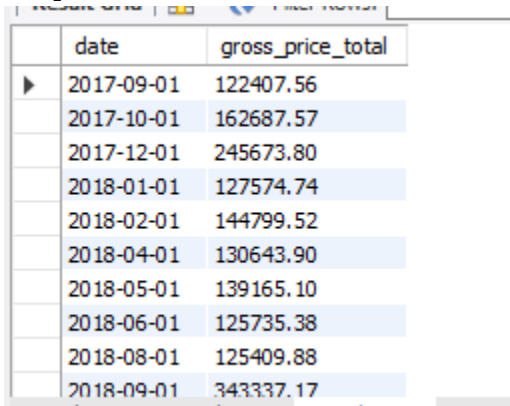
Approach:

Join sales and gross price tables, calculate gross sales amount and aggregate the results by month.

SQL Query:

```
28 • SELECT
29     s.date,
30     ROUND(SUM(g.gross_price * s.sold_quantity),2) as gross_price_total
31 from fact_sales_monthly s
32 JOIN fact_gross_price g
33 ON
34     g.product_code = s.product_code and
35     g.fiscal_year = get_fiscal_year(s.date)
36 where
37     customer_code = 90002002
38 group by s.date
39 order by date asc;
```

Output Screenshot:



date	gross_price_total
2017-09-01	122407.56
2017-10-01	162687.57
2017-12-01	245673.80
2018-01-01	127574.74
2018-02-01	144799.52
2018-04-01	130643.90
2018-05-01	139165.10
2018-06-01	125735.38
2018-08-01	125409.88
2018-09-01	343337.17

Task 7 - Yearly Gross Sales Report for Croma India using Fiscal Year

Business Objective:

Calculate the total gross sales for Croma India for each fiscal year.

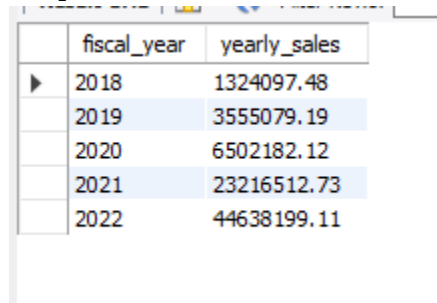
Approach:

Join sales and gross price tables, calculate gross sales, and aggregate the results by fiscal year.

SQL Query:

```
1 • select
2     get_fiscal_year(date) as fiscal_year,
3     sum(round(sold_quantity*g.gross_price,2)) as yearly_sales
4 from fact_sales_monthly s
5 join fact_gross_price g
6 on
7     g.fiscal_year=get_fiscal_year(s.date) and
8     g.product_code=s.product_code
9 where
10     customer_code=90002002
11 group by get_fiscal_year(date)
12 order by fiscal_year;
```

Output Screenshot:



	fiscal_year	yearly_sales
▶	2018	1324097.48
	2019	3555079.19
	2020	6502182.12
	2021	23216512.73
	2022	44638199.11

STORED PROCEDURE

Task 8 - Stored Procedure for Monthly Gross Sales Report

Business Objective:

Create a reusable stored procedure to generate monthly gross sales reports for any customer.

Approach:

Create a stored procedure that accepts customer code as a parameter and returns monthly gross sales data.

SQL Query:

```

1 CREATE DEFINER='root'@'localhost' PROCEDURE `get_monthly_gross_sales_for_customer`(
2     in_customer_code TEXT
3 )
4 BEGIN
5     SELECT
6         s.date,
7         SUM(s.sold_quantity * g.gross_price) as gross_price_total
8     FROM fact_sales_monthly s
9     JOIN fact_gross_price g
10    ON
11        s.product_code = g.product_code AND
12        g.fiscal_year = get_fiscal_year(s.date)
13    WHERE
14        find_in_set(s.customer_code, in_customer_code)>0
15    GROUP BY s.date
16    ORDER BY s.date asc;
17 END

```

Then call this stored procedure to get Gross Monthly Sales for Croma using its customer code.

CALL gdb0041.get_monthly_gross_sales_for_customer('90002002');

Output Screenshot:

date	gross_price_total
2017-09-01	122407.5582
2017-10-01	162687.5716
2017-12-01	245673.8042
2018-01-01	127574.7372
2018-02-01	144799.5182
2018-04-01	130643.8976
2018-05-01	139165.0975
2018-06-01	125735.3786
2018-08-01	125409.8801
2018-09-01	343337.1651

Sometimes a customer has two or more codes, so the stored procedure needs to accommodate this need. For example, let's say Amazon has two product codes so we create a stored procedure for it:


```

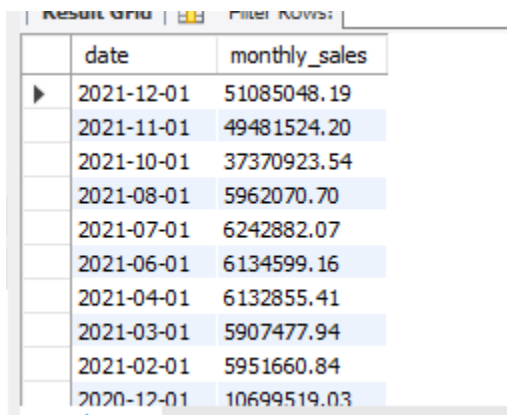
1 CREATE DEFINER='root'@'localhost' PROCEDURE `get_monthly_gross_sales_for_amazon` (
2     in_customer_codes TEXT)
3 BEGIN
4     SELECT s.date,
5         SUM(ROUND(s.sold_quantity*g.gross_price,2)) AS monthly_sales
6     FROM fact_sales_monthly s
7     JOIN fact_gross_price g
8     ON g.fiscal_year=get_fiscal_year(s.date)
9     AND g.product_code=s.product_code
10    WHERE FIND_IN_SET(s.customer_code, in_customer_codes) > 0
11    GROUP BY s.date
12    ORDER BY s.date DESC;
13 END

```

We can then call the stored procedure using the two product codes for Amazon:

CALL gdb0041.get_monthly_gross_sales_for_amazon('90002016,90002008');

Output Screenshot:



The screenshot shows a table with two columns: 'date' and 'monthly_sales'. The data is sorted in descending order by date. The first row is 2021-12-01 with a sales value of 51085048.19. The last row is 2020-12-01 with a sales value of 10699519.03.

date	monthly_sales
2021-12-01	51085048.19
2021-11-01	49481524.20
2021-10-01	37370923.54
2021-08-01	5962070.70
2021-07-01	6242882.07
2021-06-01	6134599.16
2021-04-01	6132855.41
2021-03-01	5907477.94
2021-02-01	5951660.84
2020-12-01	10699519.03

Task 9 - Stored Procedure for Market Badge Classification

Business Objective:

Assign a Gold or Silver badge to a market based on its total sales quantity in a given fiscal year.

Approach:

Join the sales and customer tables, calculate total sold quantity for a market, and use conditional logic to assign a badge.

SQL Query:

```
1 • CREATE DEFINER=`root`@`localhost` PROCEDURE `get_market_badge` (  
2     IN in_market varchar(45),  
3     IN in_fiscal_year year,  
4     OUT out_badge varchar(45)  
5 )  
6 BEGIN  
7     DECLARE qty INT default 0;  
8  
9     # set default market to be India  
10    IF in_market = "" THEN  
11        SET in_market = "India";  
12    END IF;  
14    # retrieve total qty for a given market+fyear  
15    SELECT  
16        sum(sold_quantity) into qty  
17    FROM fact_sales_monthly s  
18    JOIN dim_customer c  
19    ON s.customer_code = c.customer_code  
20    WHERE c.market = in_market AND  
21        get_fiscal_year(s.date) = in_fiscal_year  
22    GROUP BY c.market;  
23  
24    # determine market badge  
25    IF qty > 5000000 THEN  
26        SET out_badge = "Gold";  
27    ELSE  
28        SET out_badge = "Silver";  
29    END IF;  
30 END
```

Now, upon calling this stored procedure with Market = Australia and Fiscal Year = 2020, we get the output as 'Silver'. This is correct because Australia's total sales for that year are 917,945.

```

1 • set @out_badge = '0';
2 • call gdb0041.get_market_badge('Australia', 2020, @out_badge);
3 • select @out_badge;
4

```

Output Screenshot:

@out_badge
Silver

To verify the result, we can run the following query, which returns the total sales quantity for each market in the specified fiscal year.

```

1 • SELECT c.market, SUM(sold_quantity) AS total_qty
2 FROM fact_sales_monthly s
3 JOIN dim_customer c
4 ON s.customer_code = c.customer_code
5 WHERE get_fiscal_year(s.date) = 2020
6 GROUP BY c.market;
7

```

Output:

market	total_qty
India	5381959
Indonesia	576133
Japan	202306
Pakistan	427543
Philippines	1160524
South Korea	1970860
Australia	917945
Newzealand	275390

From the output, we can confirm that Australia's total sales quantity for FY 2020 is 917,945, which validates that the badge returned by the stored procedure is correct.

Task 10 - Pre-Invoice Discount Analysis Using Fiscal Year

Business Objective:

Retrieve pre-invoice discount percentages by joining sales, pricing, product, and deduction tables using the fiscal year column.

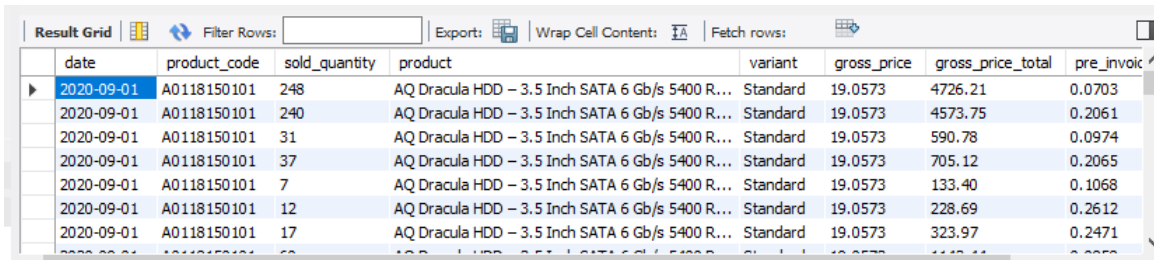
Approach:

Use the fiscal_year column in the fact_sales_monthly table to join with fact_gross_price and fact_pre_invoice_deductions tables and retrieve discount information.

SQL Query:

```
• SELECT
    s.date,s.product_code,s.sold_quantity,
    p.product,p.variant,
    g.gross_price,
    ROUND(g.gross_price * s.sold_quantity,2) as gross_price_total,
    pre.pre_invoice_discount_pct
from fact_sales_monthly s
JOIN dim_product P
    ON s.product_code = p.product_code
JOIN fact_gross_price g
    ON
        g.product_code = s.product_code AND
        g.fiscal_year = s.fiscal_year
JOIN fact_pre_invoice_deductions pre
    ON
        pre.customer_code = s.customer_code AND
        pre.fiscal_year = s.fiscal_year
where
    s.fiscal_year = 2021;
```

Output Screenshot:



The screenshot shows a database query result grid with columns: date, product_code, sold_quantity, product, variant, gross_price, gross_price_total, and pre_invoic. The data is filtered for the date 2020-09-01. The first row is highlighted in blue.

	date	product_code	sold_quantity	product	variant	gross_price	gross_price_total	pre_invoic
▶	2020-09-01	A0118150101	248	AQ Dracula HDD – 3.5 Inch SATA 6 Gb/s 5400 R...	Standard	19.0573	4726.21	0.0703
	2020-09-01	A0118150101	240	AQ Dracula HDD – 3.5 Inch SATA 6 Gb/s 5400 R...	Standard	19.0573	4573.75	0.2061
	2020-09-01	A0118150101	31	AQ Dracula HDD – 3.5 Inch SATA 6 Gb/s 5400 R...	Standard	19.0573	590.78	0.0974
	2020-09-01	A0118150101	37	AQ Dracula HDD – 3.5 Inch SATA 6 Gb/s 5400 R...	Standard	19.0573	705.12	0.2065
	2020-09-01	A0118150101	7	AQ Dracula HDD – 3.5 Inch SATA 6 Gb/s 5400 R...	Standard	19.0573	133.40	0.1068
	2020-09-01	A0118150101	12	AQ Dracula HDD – 3.5 Inch SATA 6 Gb/s 5400 R...	Standard	19.0573	228.69	0.2612
	2020-09-01	A0118150101	17	AQ Dracula HDD – 3.5 Inch SATA 6 Gb/s 5400 R...	Standard	19.0573	323.97	0.2471

DATABASE VIEWS

Task 11 - Creating the sales_preinv_discount View.

Business Objective:

Create a reusable view that combines sales, product, pricing, and pre-invoice discount data for reporting and analysis.

Approach:

Create a SQL view by joining multiple tables and storing the combined dataset as a virtual table.

SQL Query:

```
1 • CREATE VIEW `sales_preinv_discount` AS
2 SELECT
3     s.date,
4     s.fiscal_year,
5     s.customer_code,
6     c.market,
7     s.product_code,
8     p.product,
9     p.variant,
10    s.sold_quantity,
11    g.gross_price AS gross_price_per_item,
12    ROUND(s.sold_quantity * g.gross_price, 2) AS gross_price_total,
13    pre.pre_invoice_discount_pct
14 FROM fact_sales_monthly s
15 JOIN dim_customer c
16     ON s.customer_code = c.customer_code
17 JOIN dim_product p
18     ON s.product_code = p.product_code
19 JOIN fact_gross_price g
20     ON g.fiscal_year = s.fiscal_year
21     AND g.product_code = s.product_code
22 JOIN fact_pre_invoice_deductions AS pre
23     ON pre.customer_code = s.customer_code
24     AND pre.fiscal_year = s.fiscal_year;
```

Use this view to generate 'net_invoice_sales' data:

```
SELECT *,
       (gross_price_total - gross_price_total * pre_invoice_discount_pct) AS
       net_invoice_sales
FROM sales_preinv_discount;
```

Output Screenshot:

	date	fiscal_y	customer_cod	market	product_code	product	variant	sold_qua	gross_price_	gross_price_total	pre_invoice_	net_invoice_sales
▶	2017-09...	2018	70002017	India	A0118150101	AQ Dracula HDD ~...	Standard	51	15.3952	785.16	0.0824	720.462816
	2017-09...	2018	70002018	India	A0118150101	AQ Dracula HDD ~...	Standard	77	15.3952	1185.43	0.2956	835.016892
	2017-09...	2018	70003181	Indon...	A0118150101	AQ Dracula HDD ~...	Standard	17	15.3952	261.72	0.0536	247.691808
	2017-09...	2018	70003182	Indon...	A0118150101	AQ Dracula HDD ~...	Standard	6	15.3952	92.37	0.2378	70.404414
	2017-09...	2018	70006157	Philiph...	A0118150101	AQ Dracula HDD ~...	Standard	5	15.3952	76.98	0.1057	68.843214
	2017-09...	2018	70006158	Philiph...	A0118150101	AQ Dracula HDD ~...	Standard	7	15.3952	107.77	0.1875	87.563125
	2017-09...	2018	70007198	South ...	A0118150101	AQ Dracula HDD ~...	Standard	29	15.3952	446.46	0.0700	415.207800
	2017-09...	2018	70007199	South ...	A0118150101	AQ Dracula HDD ~...	Standard	34	15.3952	523.44	0.2551	389.910456
	2017-09...	2018	70008158	Philiph...	A0118150101	AQ Dracula HDD ~...	Standard	22	15.3952	338.69	0.2622	246.415816

Task 12 - Creating the sales_postinv_discount View

Business Objective:

Create a reusable view to capture post-invoice discounts and prepare data for net sales calculation.

Approach:

Create a view by joining the sales_preinv_discount view with the post-invoice deductions table and calculate the post-invoice discount percentage.

SQL Query:

```
1 • CREATE VIEW `sales_postinv_discount` AS
2 SELECT
3     s.date,
4     s.fiscal_year,
5     s.customer_code,
6     s.market,
7     s.product_code,
8     s.product,
9     s.variant,
10    s.sold_quantity,
11    s.gross_price_total,
12    s.pre_invoice_discount_pct,
13    (
14        s.gross_price_total
15        - s.pre_invoice_discount_pct * s.gross_price_total
16    ) AS net_invoice_sales,
17    (
18        po.discounts_pct
19        + po.other_deductions_pct
20    ) AS post_invoice_discount_pct
21 FROM sales_preinv_discount s
22 JOIN fact_post_invoice_deductions po
23     ON po.customer_code = s.customer_code
24     AND po.product_code = s.product_code
25     AND po.date = s.date;
```

Using this view, create a report for 'net_sales':

```

SELECT *,
       (net_invoice_sales - net_invoice_sales * post_invoice_discount_pct) AS
       net_sales
FROM gdb0041.sales_postinv_discount;

```

Output Screenshot:

	date	fiscal_y	customer_c	market	product_code	product	variant	sold_q	gross_price	pre_invoice_c	net_invoice_sale	post_invoice_	net_sales
▶	2017-09...	2018	90027207	Brazil	A0118150101	AQ Dracula HD...	Standard	4	61.58	0.2803	44.319126	0.3905	27.0125072970
	2017-11...	2018	90027207	Brazil	A0118150101	AQ Dracula HD...	Standard	16	246.32	0.2803	177.276504	0.4139	103.9017589944
	2017-12...	2018	90027207	Brazil	A0118150101	AQ Dracula HD...	Standard	4	61.58	0.2803	44.319126	0.3295	29.7159739830
	2018-01...	2018	90027207	Brazil	A0118150101	AQ Dracula HD...	Standard	6	92.37	0.2803	66.478689	0.3244	44.9130022884
	2018-03...	2018	90027207	Brazil	A0118150101	AQ Dracula HD...	Standard	9	138.56	0.2803	99.721632	0.3766	62.1664653888
	2018-04...	2018	90027207	Brazil	A0118150101	AQ Dracula HD...	Standard	6	92.37	0.2803	66.478689	0.3615	42.4466429265
	2018-05...	2018	90027207	Brazil	A0118150101	AQ Dracula HD...	Standard	7	107.77	0.2803	77.562069	0.3173	52.9516245063

Now save 'net_sales' as a view for future use:

```

1 • CREATE VIEW `net_sales` AS
2   SELECT
3     *,
4     net_invoice_sales * (1 - post_invoice_discount_pct) AS net_sales
5   FROM sales_postinv_discount;

```

Task 13 - Get Top 5 Market (Countries) by Net Sales in Fiscal Year 2021

Business Objective:

Identify the top-performing markets based on net sales for a selected fiscal year(2021)

Approach:

The result is calculated by aggregating net sales at market level, converting it into millions, and sorting in descending order.

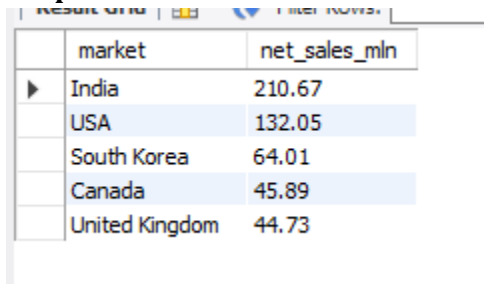
SQL Query:

```

• SELECT market,
      ROUND(sum(net_sales)/1000000,2) AS net_sales_mln
FROM gdb0041.net_sales
WHERE fiscal_year = 2021
GROUP BY market
ORDER BY net_sales_mln DESC
LIMIT 5;

```

Output Screenshot:



The screenshot shows a database query result with two columns: 'market' and 'net_sales_mln'. The results are ordered by net sales in descending order. The top 5 markets are India (210.67), USA (132.05), South Korea (64.01), Canada (45.89), and United Kingdom (44.73).

market	net_sales_mln
India	210.67
USA	132.05
South Korea	64.01
Canada	45.89
United Kingdom	44.73

Task 14 - Dynamic Stored Procedure for Top N Markets by Net Sales.

Business Objective:

The above query is used to identify the top 5 markets based on net sales for a given fiscal year (currently hardcoded as 2021 and top 5). Convert that logic into a stored procedure to make it dynamic and reusable.

Approach:

Convert the existing query into a stored procedure by passing the fiscal year and the number of top markets (N) as input parameters instead of hardcoding values.

SQL Query:

```
CREATE DEFINER='root'@'localhost' PROCEDURE `get_top_n_markets_by_netsales`(  
    IN in_fiscal_year INT,  
    IN in_top_n INT  
)  
BEGIN  
    SELECT market,  
           ROUND(sum(net_sales)/1000000,2) AS net_sales_mln  
    FROM gdb0041.net_sales  
    WHERE fiscal_year = in_fiscal_year  
    GROUP BY market  
    ORDER BY net_sales_mln DESC  
    LIMIT in_top_n;  
END
```

Use this stored procedure to see Top 5 markets in 2020:

CALL gdb0041.get_top_n_markets_by_netsales(2020, 5);

Output Screenshot:

	market	net_sales_mln
▶	India	64.73
	USA	46.35
	South Korea	22.38
	Philippines	17.45
	Canada	15.87

Task 15 - Create a stored procedure to get Top N Customers by Net Sales for a chosen year.

Business Objective:

Create a reusable stored procedure to identify the top N customers based on net sales for a selected fiscal year.

SQL Query:

```
1 • CREATE DEFINER=`root`@`localhost` PROCEDURE `get_top_n_customers_by_netsales`(  
2     IN in_market VARCHAR(45),  
3     IN in_fiscal_year INT,  
4     IN in_top_n INT  
5 )  
6 BEGIN  
7     SELECT c.customer,  
8         ROUND(sum(net_sales)/1000000,2) AS net_sales_mln  
9     FROM gdb0041.net_sales n  
10    JOIN dim_customer c  
11        ON c.customer_code = n.customer_code  
12    WHERE n.fiscal_year = in_fiscal_year AND  
13          n.market = in_market  
14    GROUP BY c.customer  
15    ORDER BY net_sales_mln DESC  
16    LIMIT in_top_n;  
17 END
```

Call this stored procedure to see Top 5 customers in USA in 2020:

CALL gdb0041.get_top_n_customers_by_netsales('usa', 2020, 5);

Output Screenshot:

	customer	net_sales_mln
▶	Amazon	10.00
	Flipkart	4.95
	Ebay	3.45
	Atliq e Store	3.42
	Atliq Exclusive	3.08

COMMON TABLE EXPRESSION & WINDOW FUNCTION

Task 16 - Find out customer-wise Net Sales percentage contribution(globally) for FY 2021

Business Objective:

Calculate the percentage contribution of each customer to total net sales in FY 2021.

Approach:

Aggregate net sales by customer and use a window function to calculate each customer's percentage contribution to overall net sales.

SQL Query:

```
1 WITH cte1 AS (  
2     SELECT c.customer,  
3           ROUND(sum(net_sales)/1000000,2) AS net_sales_mln  
4     FROM gdb0041.net_sales n  
5     JOIN dim_customer c  
6       ON c.customer_code = n.customer_code  
7     WHERE n.fiscal_year = 2021  
8     GROUP BY c.customer)  
9 SELECT *,  
10    net_sales_mln*100/sum(net_sales_mln) OVER() as net_sales_pct  
11 FROM cte1  
12 ORDER BY net_sales_mln DESC
```

Output Screenshot:

	customer	net_sales_mln	net_sales_pct
▶	Amazon	109.03	13.233402
	Atliq Exclusive	79.92	9.700206
	Atliq e Store	70.31	8.533803
	Sage	27.07	3.285593
	Flipkart	25.25	3.064692
	Leader	24.52	2.976089
	Neptune	21.01	2.550067
	Ebay	19.88	2.412914

Task 17 - Find Customer-wise Net Sales distribution per region for Fiscal Year 2021.

Business Objective:

Calculate the percentage contribution of each customer to total net sales within their respective region for FY 2021.

Approach:

Aggregate net sales by customer and region, then use a window function with PARTITION BY region to calculate each customer's regional market share.

SQL Query:

```
1 • WITH cte1 AS (  
2   SELECT c.customer,  
3         c.region,  
4         ROUND(sum(net_sales)/1000000,2) AS net_sales_mln  
5   FROM gdb0041.net_sales n  
6   JOIN dim_customer c  
7        ON c.customer_code = n.customer_code  
  
8   WHERE n.fiscal_year = 2021  
9   GROUP BY c.customer,c.region  
10  )  
11  SELECT *,  
12         net_sales_mln*100/sum(net_sales_mln) OVER(partition by region) AS pct  
13  FROM cte1  
14  ORDER BY net_sales_mln DESC
```

Output Screenshot:

	customer	region	net_sales_mln	pct
▶	Amazon	APAC	57.41	12.988688
	Atliq Exclusive	APAC	51.58	11.669683
	Atliq e Store	APAC	36.97	8.364253
	Amazon	NA	30.31	17.033832
	Leader	APAC	24.52	5.547511
	Sage	APAC	22.85	5.169683
	Neptune	APAC	21.01	4.753394
	Atliq e Store	EU	19.83	9.874514

Task 18 - Identify the top-performing products in each division based on total quantity sold for a selected fiscal year.

Used CTEs to calculate total quantity sold for each product and apply DENSE_RANK() to rank products within each division.

SQL Query:

```

1 CREATE DEFINER=`root`@`localhost` PROCEDURE `get_top_n_products_per_division_by_qty_sold` (
2     IN in_fiscal_year INT,
3     IN in_limit INT
4 )
5 BEGIN
6     WITH cte1 AS (
7         SELECT p.division,
8                p.product,
9                s.sold_quantity,
10               sum(s.sold_quantity) as total_quantity
11         FROM fact_sales_monthly s
12         JOIN dim_product p
13         ON
14            p.product_code = s.product_code
15
16         WHERE s.fiscal_year = in_fiscal_year
17         GROUP BY p.product),
18     cte2 AS (
19         SELECT *,
20                dense_rank() over(partition by division order by total_quantity desc) as drank
21         FROM cte1)
22     SELECT *FROM cte2
23     WHERE drank<=in_limit;
24 END

```

Call this stored procedure to see Top 5 best performing products in 2020 based on total quantity sold:

CALL gdb0041.get_top_n_products_per_division_by_qty_sold(2020, 10);

Output Screenshot:

	division	product	sold_quantity	total_quantity	drank
▶	N & S	AQ Clx1	318	935128	1
	N & S	AQ Neuer SSD	423	924264	2
	N & S	AQ Digit SSD	613	920105	3
	N & S	AQ Wi Power Dx2	122	846576	4
	N & S	AQ Wi Power Dx1	524	844664	5
	P & A	AQ Master wired x1 Ms	498	1578253	1
	P & A	AQ Gamers Ms	373	1566445	2
	P & A	AQ Lite Ms	253	1564099	3
	P & A	AQ Master wireless x1 Ms	128	1563844	4
	P & A	AQ Gamers	522	1263573	5
	PC	AQ Digit	6	68862	1
	PC	AQ Elite	8	67841	2
	PC	AQ Aspireon	15	59516	3
	PC	AQ BZ Compact	5	52380	4
	PC	AQ BZ Gen Y	15	52047	5

Task 19 - Retrieve the Top 2 Markets in every Region by their gross sales amount in Fiscal Year 2021.

SQL Query:

```
1 • WITH cte1 AS (  
2     SELECT c.market,  
3           c.region,  
4           ROUND(SUM(g.gross_price_total)/1000000,2) AS gross_sales_mln  
5     FROM dim_customer c  
6     JOIN gross_sales g  
7     ON g.customer_code = c.customer_code  
8  
9     WHERE g.fiscal_year = 2021  
10    GROUP BY c.market),  
11    cte2 AS (  
12        SELECT *,  
13        dense_rank() over(partition by region order by gross_sales_mln desc) AS drank  
14    FROM cte1)  
15    SELECT * FROM cte2 WHERE drank <= 2
```

Output Screenshot:

	market	region	gross_sales_mln	drank
▶	India	APAC	455.05	1
	South Korea	APAC	131.86	2
	United Kingdom	EU	78.11	1
	France	EU	67.62	2
	Mexico	LATAM	2.30	1
	Brazil	LATAM	2.14	2
	USA	NA	264.46	1
	Canada	NA	89.78	2